Technical report ? Apertus Draft Gate

- Track: Track 2B ? Own Project
- Event: Hack Apertus Online Hackathon, October 2026
- Team: Nick Menzi
- Demo: Local interactive Docker demo (make run) and offline captioned overview at docs/Apertus_Draft_Gate_Demo.mp4; no public video URL is included in this repository.

1. Summary

Apertus Draft Gate is a small, safety-first prototype for a controlled email-marketing draft workflow. make run opens an isolated Hermes terminal session configured for an operator-selected Apertus-compatible endpoint; make gateway is reserved for a separately configured persistent messaging gateway. The deterministic draft path places a policy gate before an Apertus request: it sends only campaign-approved fictional fields, keeps the recipient address and internal note outside the model context, and produces a draft that remains pending human approval. The implementation deliberately has no email-send, CRM-update or messaging capability.

2. Architecture

The Docker service uses scripts/start-hermes.sh to create an isolated project-local Hermes home at /opt/data and copy a non-secret configuration template only on first start. It reads the operator’s LLM_NAME, LLM_BASE_URL and LLM_API_KEY from the untracked local .env file. make run invokes that bootstrap and starts hermes --tui, so a judge can interact with the demonstrator immediately. make gateway invokes the same bootstrap without a TUI and starts hermes gateway run --no-supervise --external-supervisor for a separately configured messaging platform. The regular host Hermes profile is never mounted.

The deterministic make draft path retains the fixture, pure-Python policy module and explicit confirmation gate. policy.py validates an allowlist and rejects email, id and internal_note even if a faulty campaign configuration lists them. workflow.py first creates a preflight record, then ? only after the operator answers y ? posts the minimised JSON context to an OpenAI-compatible Apertus /chat/completions endpoint. The result is printed together with the original data-minimisation evidence, no_send: true, and a pending approval record.

The deterministic model request contains a constrained drafting instruction and the minimised fictional context. It does not contain the recipient email, record ID, internal note or a send instruction. There is no SMTP client, CRM SDK, webhook, queue or external-action tool in the codebase.

Target architecture (mandatory)

a) On-premise. The container is deployable with an organisation-controlled, OpenAI-compatible Apertus inference endpoint set in LLM_BASE_URL. The fixture, policy and approval logic remain local. At runtime, the only optional external dependency is that configured model endpoint, and only after explicit operator confirmation. The same configuration pattern can also target an organisation-approved Swiss-hosted inference endpoint; this repository does not claim to operate such infrastructure.

3. Use of Apertus

- Model: configured through LLM_NAME; .env.example uses swiss-ai/Apertus-v1.5-8B.
- How it is used: inference for a concise German invitation-email draft.
- Where it runs: an OpenAI-compatible endpoint configured by the operator through LLM_BASE_URL and LLM_API_KEY.
- Prompt boundary: a fixed system instruction prohibits invented facts and sending claims. The user message is generated with json.dumps() from the allowlisted model context only.

The project does not fine-tune Apertus, collect prompts, retain model outputs or use tools for external actions.

4. Data

data/test_contacts.json contains two deliberately fictional contacts and one fictional campaign. It is the only input dataset. The fixture includes email and internal-note fields specifically to test that they are excluded from the model payload. No real customer, mailbox, CRM, payment, account or contact data is used or required. The data directory is far below the 100 MB limit.

5. Evaluation

The prototype is evaluated as a safety-contract test, not as a semantic-quality benchmark. The local deterministic suite was run with python3 -m unittest discover -s tests -v.

Setup	Metric	Result
Policy gate	Forbidden model fields rejected	1/1 test passed
Policy gate	Approved fields only; excluded fields reported	2/2 tests passed
Preflight	Known contact, approval-pending and no-send states	3/3 tests passed
Draft request	Recipient, ID and internal note absent from payload	1/1 test passed
Draft request	Invalid model response rejected	1/1 test passed
Live CLI	Model endpoint failure returned as no-send error record	1/1 test passed
Container / gateway contract	Isolated persistent Hermes runtime, chat and explicit draft commands	6/6 tests passed
Total	Deterministic automated tests	15/15 passed

The live Apertus endpoint was intentionally not invoked during this repository verification because no credentials or external endpoint are included in the source tree. The tested request seam uses a local fake response to prove payload construction and no-send state handling without transmitting data.

6. Limitations

This is a narrow prototype, not a production mail system. It does not measure factual quality, tone, language quality or hallucination rate of a live Apertus model. A model can still write an unsuitable draft, so exact draft, recipient and purpose remain pending human approval. The implementation does not persist approvals or drafts, authenticate human reviewers, support multiple campaigns, render HTML email or integrate a real contact system.

The endpoint operator must secure LLM_API_KEY, provide the selected Apertus model and apply network controls appropriate to their deployment.

7. Reproducibility

Requirements: Docker with Docker Compose for the container modes, or Python 3.12+ for the offline tests. Copy .env.example to .env, configure an Apertus endpoint and run make run from track_2b/ to start the interactive isolated Hermes session. make chat is an alias; make gateway starts the persistent gateway only when a messaging platform has been separately configured. Use make draft for the deterministic policy preflight; only entering y in the latter performs one live model request with fictional minimised data. Run the deterministic checks with make test. The report reflects branch feat/run-to-finish; the exact source revision is recorded in Git history.

8. Next steps

With more time, the project would add a review UI that binds an approval to a draft hash, recipient and campaign purpose; structured red-team cases for prompt injection and unsafe claims; an offline/on-premise model deployment recipe; a human-rated, licensed evaluation set; and an immutable audit log that contains field names and approval events but never unneeded contact content.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.